Lista de Exercícios 2: Orientação a Objetos em Python

UniAcademia, Juiz de Fora/MG · Prof. Marcos Miguel

Orientações gerais

Esta lista tem 8 exercícios em ordem crescente de dificuldade, do básico ao avançado, e vale **20 pontos**. Ela reúne o que o Trabalho Final vai exigir: datas, listas de objetos, herança e classes abstratas, leitura de arquivos, tratamento de exceções e agrupamento de dados. Faça na ordem, porque cada bloco usa o anterior.

Entrega: 30/11, até as 23:59.

Bloco	Tema	Exercícios
1	Classe, regras e datas	1 e 2
2	Associação, datas com multa e herança	3, 4 e 5
3	Arquivos e login	6 e 7
Desafio	Integrador	8

Regras de todos os exercícios

- Crie um único projeto para todos os exercícios, com uma pasta por exercício: `Exercicio1`, `Exercicio2`, até `Exercicio8`. Cada pasta tem o seu próprio `main.py` e uma subpasta `model/` com uma classe por arquivo (por exemplo `Exercicio1/model/Ingresso.py`). Cada `main.py` roda sozinho, sem depender de outro exercício.
- No `main.py`, a função `principal()` testa as classes e imprime os resultados, e é chamada dentro de `if __name__ == '__main__':`.
- Classes em PascalCase e métodos em camelCase.
- Atributos privados com dois sublinhados (`self.__nome`), acessados por `getNome()` e `setNome()`. Os métodos `get` e `set` não aparecem nas tabelas de classes: crie conforme a necessidade.
- Métodos que incluem ou retiram elementos de uma lista começam com `add` e `remove`.

- Para exibir um objeto, use `__str__` ou um método `toString()`.
- Datas com `datetime` e `strptime`. Arquivos com `with open`. Erros com `try/except/finally` e exceções como `ValueError`.
- As tabelas de classes são ponto de partida: pode acrescentar atributos e métodos que a solução exigir.
- Cada exercício traz um exemplo de entrada e saída. Seu código deve produzir exatamente aquela saída.
- Pode fazer em dupla. Pode consultar material, mas esteja pronto para explicar cada linha que escreveu.
- Entrega em tarefa própria na Plataforma A, no prazo.

Distribuição dos 20 pontos

Exercício	Tema	Pontos
1	Ingresso de cinema	1,5
2	Estacionamento	2,0
3	Playlist	2,0
4	Mensalidade com multa	2,5
5	Assinaturas de streaming	2,5
6	Estoque em arquivo	3,0
7	Login com tipos de usuário	3,0
8	Desafio: locação de equipamentos	3,5
Total		**20,0**

Bloco 1: classe, regras e datas

Exercício 1: Ingresso de cinema (1,5 ponto)

Objetivo: criar uma classe com atributos privados, `get` / `set` e um método com regras, lançando `ValueError` para valores inválidos.

Classe	Atributos	Métodos
`Ingresso`	`-filme: str`, `-valorBase: float`	`+calcularValor(tipo): float`

1. O construtor recebe o filme e o valor base. `setValorBase` só aceita valor maior que zero; caso contrário, lança `ValueError("Valor base deve ser maior que zero")`.
2. `calcularValor(tipo)` devolve o valor do ingresso: `inteira` paga o valor base, `meia` paga 50% e `idoso` paga 40%.
3. Para qualquer outro tipo, lança `ValueError("Tipo de ingresso inválido: vip")`, com o tipo recebido no lugar de `vip`.

Exemplo de entrada e saída:

```
i = Ingresso("Duna", 40.0)
print(i.calcularValor("inteira"))
print(i.calcularValor("meia"))
print(i.calcularValor("idoso"))
try:
    i.calcularValor("vip")
except ValueError as erro:
    print(erro)
try:
    i.setValorBase(-5.0)
except ValueError as erro:
    print(erro)
```

```
40.0
20.0
16.0
Tipo de ingresso inválido: vip
Valor base deve ser maior que zero
```

Para quem terminar cedo: aceite o tipo em maiúsculas ou minúsculas (`Meia`, `MEIA`) e guarde os tipos e percentuais em um dicionário, em vez de vários `if`.

Exercício 2: Estacionamento (2,0 pontos)

Objetivo: trabalhar com `datetime`, diferença entre datas e regras de cobrança com limite.

Classe	Atributos	Métodos
`Ticket`	`-placa: str`, `-entrada: datetime`	`+permanencia(saida): int`, `+calcularValor(saida): float`

1. `permanencia(saida)` devolve o tempo estacionado em minutos inteiros.
2. `calcularValor(saida)` segue a tabela abaixo, na ordem.
3. Se a saída for anterior à entrada, lança `ValueError("Saída anterior à entrada")`.

Permanência	Valor
até 15 minutos	R$ 0,00
de 16 a 60 minutos	R$ 8,00
acima de 60 minutos	R$ 8,00 mais R$ 4,00 por hora adicional ou fração de hora
qualquer permanência	no máximo R$ 40,00

Exemplo de entrada e saída:

```
from datetime import datetime

formato = '%d/%m/%Y %H:%M'
t = Ticket("ABC1234", datetime.strptime('21/10/2026 10:00', formato))
saidas = ['21/10/2026 10:10', '21/10/2026 10:45', '21/10/2026 11:00',
          '21/10/2026 11:01', '21/10/2026 13:30', '22/10/2026 10:00']
for texto in saidas:
    saida = datetime.strptime(texto, formato)
    print(f"{texto}: {t.permanencia(saida)} min, R$
{t.calcularValor(saida):.2f}")
try:
    t.calcularValor(datetime.strptime('21/10/2026 09:00', formato))
except ValueError as erro:
    print(erro)
```

```
21/10/2026 10:10: 10 min, R$ 0.00
21/10/2026 10:45: 45 min, R$ 8.00
21/10/2026 11:00: 60 min, R$ 8.00
21/10/2026 11:01: 61 min, R$ 12.00
21/10/2026 13:30: 210 min, R$ 20.00
22/10/2026 10:00: 1440 min, R$ 40.00
Saída anterior à entrada
```

Dica: `(saida - entrada).total_seconds() / 60` dá os minutos, e `math.ceil` arredonda a fração de hora para cima. Confira os casos de fronteira: 15, 60 e 61 minutos.

Bloco 2: associação, multa por atraso e herança

Exercício 3: Playlist (2,0 pontos)

Objetivo: associação 1 para n (uma playlist tem várias músicas), métodos `add` e `remove` e consultas sobre uma lista de objetos.

Classe	Atributos	Métodos
`Musica`	`-titulo: str`, `-artista: str`, `-duracao: int` (segundos)	
`Playlist`	`-nome: str`, `-musicas: list`	`+addMusica(musica): bool`, `+removeMusica(titulo): bool`, `+duracaoTotal(): str`, `+musicaMaisLonga(): Musica`, `+musicasDoArtista(artista): list`

1. `addMusica` não aceita música repetida (mesmo título e mesmo artista): devolve `False` nesse caso e `True` quando adiciona.
2. `removeMusica` tira a música pelo título e devolve `True`, ou `False` se não existir.
3. `duracaoTotal` soma as durações e devolve o texto no formato `mm:ss`.
4. `musicaMaisLonga` devolve o objeto da música mais longa, ou `None` se a playlist estiver vazia.
5. `musicasDoArtista` devolve a lista com os títulos das músicas do artista.

Exemplo de entrada e saída:

```
p = Playlist("Bossa Nova")
print(p.addMusica(Musica("Garota de Ipanema", "Tom Jobim", 190)))
print(p.addMusica(Musica("Wave", "Tom Jobim", 198)))
print(p.addMusica(Musica("Aquarela", "Toquinho", 215)))
print(p.addMusica(Musica("Wave", "Tom Jobim", 198)))
print(p.duracaoTotal())
print(p.musicaMaisLonga().getTitulo())
print(p.musicasDoArtista("Tom Jobim"))
print(p.removeMusica("Wave"))
print(p.removeMusica("Wave"))
print(p.duracaoTotal())
```

```
True
True
True
False
10:03
Aquarela
['Garota de Ipanema', 'Wave']
True
False
06:45
```

Para quem terminar cedo: se a duração passar de uma hora, mostre `h:mm:ss`.

Exercício 4: Mensalidade com multa por atraso (2,5 pontos)

Objetivo: cálculo de multa com datas, estado do objeto e associação 1 para n.

Classe	Atributos	Métodos
`Aluno`	`-nome: str`, `-mensalidades: list`	`+addMensalidade(mensalidade)`, `+totalEmAberto(dataAtual): float`
`Mensalidade`	`-valor: float`, `-vencimento: datetime`, `-pago: bool`	`+diasAtraso(data): int`, `+calcularTotal(data): float`, `+situacao(data): str`, `+pagar(data): float`

1. `diasAtraso(data)` devolve quantos dias a data passou do vencimento, ou 0 se estiver em dia.

2. `calcularTotal(data)` devolve o valor da mensalidade. Em atraso, soma multa de 2% do valor e R$ 1,50 por dia de atraso.
3. `situacao(data)` devolve `em dia` ou `atrasada há N dias`.
4. `pagar(data)` marca a mensalidade como paga e devolve o total pago. Se já estiver paga, lança `ValueError("Mensalidade já paga")`.
5. `totalEmAberto(dataAtual)` soma o total (com multa) das mensalidades ainda não pagas.

Exemplo de entrada e saída:

```
from datetime import datetime

def data(texto):
    return datetime.strptime(texto, '%d/%m/%Y')

hoje = data('20/10/2026')
ana = Aluno("Ana")
ana.addMensalidade(Mensalidade(200.0, data('10/10/2026')))
ana.addMensalidade(Mensalidade(200.0, data('10/11/2026')))
ana.addMensalidade(Mensalidade(200.0, data('10/09/2026')))

for m in ana.getMensalidades():
    print(f"{m.situacao(hoje)}: R$ {m.calcularTotal(hoje):.2f}")
print(f"Total em aberto: R$ {ana.totalEmAberto(hoje):.2f}")

primeira = ana.getMensalidades()[0]
print(f"Pago: R$ {primeira.pagar(data('12/10/2026')):.2f}")
print(f"Total em aberto: R$ {ana.totalEmAberto(hoje):.2f}")
try:
    primeira.pagar(hoje)
except ValueError as erro:
    print(erro)
```

```
atrasada há 10 dias: R$ 219.00
em dia: R$ 200.00
atrasada há 40 dias: R$ 264.00
Total em aberto: R$ 683.00
Pago: R$ 207.00
Total em aberto: R$ 464.00
Mensalidade já paga
```

Dica: `(data - vencimento).days` dá a diferença em dias. Arredonde o total com `round(valor, 2)`.

Exercício 5: Assinaturas de streaming (2,5 pontos)

Objetivo: herança, classe abstrata e polimorfismo.

Classe	Atributos	Métodos
`Assinatura` (abstrata)	`-assinante: str`	`+calcularMensalidade(): float` (abstrato), `+nomePlano(): str` (abstrato), `+descricao(): str`
`PlanoBasico`		
`PlanoPadrao`	`-telas: int`	
`PlanoPremium`	`-mesesContrato: int`	

1. Declare `Assinatura` como classe abstrata (`abc.ABC`), com `calcularMensalidade` e `nomePlano` abstratos. Criar `Assinatura(...)` diretamente deve dar `TypeError`.
2. `descricao()` fica na classe mãe e usa os dois métodos abstratos, no formato `Ana: Plano Básico, R$ 20.00`.
3. `PlanoBasico`: R$ 20,00 por mês.
4. `PlanoPadrao`: R$ 30,00 com até 2 telas, mais R$ 5,00 por tela adicional. Aceita de 1 a 4 telas; fora disso, lança `ValueError("Número de telas deve ser de 1 a 4")`.
5. `PlanoPremium`: R$ 45,00 por mês, com 10% de desconto se o contrato for de 12 meses ou mais.

Exemplo de entrada e saída:

```
planos = [PlanoBasico("Ana"), PlanoPadrao("Bia", 4),
          PlanoPremium("Caio", 12), PlanoPremium("Davi", 1)]
for plano in planos:
    print(plano.descricao())
print(f"Receita mensal: R$ {sum(p.calcularMensalidade() for p in
planos):.2f}")
try:
    Assinatura("X")
except TypeError:
    print("Assinatura é abstrata")
try:
    PlanoPadrao("Eva", 5)
except ValueError as erro:
    print(erro)
```

```
Ana: Plano Básico, R$ 20.00
Bia: Plano Padrão, R$ 40.00
Caio: Plano Premium, R$ 40.50
Davi: Plano Premium, R$ 45.00
Receita mensal: R$ 145.50
Assinatura é abstrata
Número de telas deve ser de 1 a 4
```

Perguntas para discutir: o laço `for` chama `descricao()` sem saber o tipo de cada plano. Como o Python escolhe a mensalidade certa? E por que `descricao()` pode ficar na classe mãe?

Bloco 3: arquivos e login

Exercício 6: Estoque em arquivo (3,0 pontos)

Objetivo: ler e gravar arquivos `.txt` com `with open`, tratar linhas inválidas e arquivo inexistente.

Classe	Atributos	Métodos
`Produto`	`-codigo: int`, `-nome: str`, `-preco: float`, `-quantidade: int`	`+valorEmEstoque(): float`

Classe	Atributos	Métodos
`Estoque`	`-produtos: list`	`+addProduto(produto)`, `+carregarArquivo(nomeArquivo): int`, `+valorTotal(): float`, `+produtoMaisCaro(): Produto`, `+gravarRelatorio(nomeArquivo)`

O arquivo `produtos.txt` tem um produto por linha, com os campos separados por ponto e vírgula: código, nome, preço e quantidade.

```
1;Caneta;2.50;100
2;Caderno;18.90;40
3;Mochila;texto;10
4;Lapis;1.20
5;Borracha;1.00;50
```

1. `carregarArquivo` lê o arquivo, cria um `Produto` para cada linha válida e devolve quantos produtos foram carregados.
2. Linha com preço que não é número: exiba `Linha N ignorada: preço inválido` e siga para a próxima. Linha com menos de 4 campos: exiba `Linha N ignorada: campos incompletos`. `N` é o número da linha no arquivo.
3. Se o arquivo não existir, o `FileNotFoundError` sobe para quem chamou, que trata o erro.
4. `valorTotal` soma o valor em estoque de todos os produtos, e `produtoMaisCaro` devolve o de maior preço.
5. `gravarRelatorio` grava um arquivo com uma linha por produto e o total no final.

Exemplo de entrada e saída:

```
estoque = Estoque()
carregados = estoque.carregarArquivo("produtos.txt")
print(f"{carregados} produtos carregados")
print(f"Produto mais caro: {estoque.produtoMaisCaro().getNome()}")
print(f"Valor total em estoque: R$ {estoque.valorTotal():.2f}")
estoque.gravarRelatorio("relatorio.txt")
print("Relatório gravado em relatorio.txt")
try:
    Estoque().carregarArquivo("inexistente.txt")
except FileNotFoundError:
    print("Arquivo não encontrado: inexistente.txt")
```

```
Linha 3 ignorada: preço inválido
Linha 4 ignorada: campos incompletos
3 produtos carregados
Produto mais caro: Caderno
Valor total em estoque: R$ 1056.00
Relatório gravado em relatorio.txt
Arquivo não encontrado: inexistente.txt
```

Conteúdo esperado de `relatorio.txt`:

```
RELATORIO DE ESTOQUE
1 - Caneta: 100 x R$ 2.50 = R$ 250.00
2 - Caderno: 40 x R$ 18.90 = R$ 756.00
5 - Borracha: 50 x R$ 1.00 = R$ 50.00
TOTAL: R$ 1056.00
```

Dicas: `linha.strip().split(";")` separa os campos. Use `enumerate(arquivo, start=1)` para numerar as linhas. Converta o preço dentro de um `try/except ValueError`.

Exercício 7: Login com tipos de usuário (3,0 pontos)

Objetivo: validação de acesso com dados de arquivo, classe abstrata e polimorfismo.

Classe	Atributos	Métodos
`Usuario` (abstrata)	`-codigo: int`, `-nome: str`, `-login: str`, `-senha: str`	`+autenticar(login, senha): bool`, `+tipo(): str` (abstrato), `+menu(): str` (abstrato)

Classe	Atributos	Métodos
`Cliente`		
`Administrador`		
`Sistema`	`-usuarios: list`	`+addUsuario(usuario)`, `+carregarArquivo(nomeArquivo)`, `+entrar(login, senha): Usuario`

O arquivo `usuarios.txt` tem: código, nome, tipo (`Cliente` ou `Administrador`), login e senha.

```
1;Ana Souza;Cliente;ana;1234
2;Bruno Lima;Administrador;bruno;admin
3;Carla Dias;Visitante;carla;abcd
```

1. `carregarArquivo` cria um `Cliente` ou um `Administrador` conforme o tipo da linha. Tipo desconhecido: exiba `Linha N ignorada: tipo desconhecido`.
2. `entrar(login, senha)` devolve o usuário cujo login e senha conferem, ou `None` se não houver.
3. O `menu()` de cada tipo é diferente. O programa chama `usuario.menu()` sem se preocupar com o tipo.

Tipo	Menu
Cliente	`1 - Ver catálogo`, `2 - Meus pedidos`, `3 - Sair`
Administrador	`1 - Ver catálogo`, `2 - Todos os pedidos`, `3 - Cadastrar usuário`, `4 - Sair`

Exemplo de entrada e saída:

```
sistema = Sistema()
sistema.carregarArquivo("usuarios.txt")
tentativas = [("ana", "1234"), ("ana", "errada"), ("bruno", "admin"),
("carla", "abcd")]
for login, senha in tentativas:
    usuario = sistema.entrar(login, senha)
    if usuario is None:
        print(f"Acesso negado para {login}")
    else:
        print(f"Bem-vindo, {usuario.getNome()} ({usuario.tipo()})")
        print(usuario.menu())
```

```
Linha 3 ignorada: tipo desconhecido
Bem-vindo, Ana Souza (Cliente)
1 - Ver catálogo
2 - Meus pedidos
3 - Sair
Acesso negado para ana
Bem-vindo, Bruno Lima (Administrador)
1 - Ver catálogo
2 - Todos os pedidos
3 - Cadastrar usuário
4 - Sair
Acesso negado para carla
```

Para quem terminar cedo: peça o login e a senha com `input()` em um laço que permite no máximo 3 tentativas, e guarde a senha como hash (`hashlib.sha256`) em vez de texto puro.

Desafio integrador

Exercício 8: Locação de equipamentos (3,5 pontos)

Objetivo: juntar arquivos, agrupamento de linhas por código, associações 1 e n, datas, multa e renovação em um único problema.

Classe	Atributos	Métodos
`Cliente`	`-codigo: int`, `-nome: str`	

Classe	Atributos	Métodos
`Equipamento`	`-codigo: int`, `-nome: str`	
`Locacao`	`-codigo: int`, `-cliente: Cliente`, `-dataRetirada: datetime`, `-equipamentos: list`, `-renovacoes: int`	`+addEquipamento(equipamento): bool`, `+dataDevolucao(): datetime`, `+diasAtraso(dataAtual): int`, `+calcularMulta(dataAtual): float`, `+renovar(dataAtual): datetime`
`Locadora`	`-clientes: list`, `-equipamentos: list`, `-locacoes: list`	`+carregarArquivos(...)`, `+listarLocacoes(codigoCliente, dataAtual)`, `+renovarLocacao(codigo, dataAtual): datetime`

Os dados ficam em três arquivos, com campos separados por ponto e vírgula.

```
clientes.txt (código;nome)
1;Ana
2;Bruno

equipamentos.txt (código;nome)
10;Câmera
20;Tripé
30;Projetor
40;Microfone

locacoes.txt (código da locação;código do cliente;código do equipamento;data de retirada)
100;1;10;10/10/2026
100;1;20;10/10/2026
101;2;30;18/10/2026
102;1;40;19/10/2026
```

Regras:

1. Uma locação pode ocupar de 1 a 3 linhas do `locacoes.txt`, uma por equipamento. Ao ler o arquivo, agrupe as linhas de mesmo código em um único objeto `Locacao`. Se aparecer uma 4ª linha para o mesmo código, ignore e exiba `Linha N ignorada: locação com mais de 3 equipamentos`.
2. O prazo de devolução é de 5 dias a partir da retirada, e cada renovação soma mais 5 dias.

3. A multa é de R$ 2,00 por dia de atraso, por equipamento.
4. Cada locação pode ser renovada uma vez. Locação atrasada não renova.
5. `renovarLocacao` lança `ValueError` com a mensagem `Locação inexistente`, `Locação atrasada, não pode renovar` ou `Limite de renovações atingido`, nessa ordem de verificação, e devolve a nova data de devolução quando renova.
6. `listarLocacoes` mostra as locações do cliente em ordem de código, no formato do exemplo, e o total de multas no final.
7. Se algum arquivo não existir, trate o `FileNotFoundError` e informe qual arquivo faltou.

Exemplo de entrada e saída:

```
from datetime import datetime

hoje = datetime.strptime('21/10/2026', '%d/%m/%Y')
locadora = Locadora()
locadora.carregarArquivos("clientes.txt", "equipamentos.txt", "locacoes.txt")
locadora.listarLocacoes(1, hoje)
locadora.listarLocacoes(2, hoje)
for codigo in [101, 101, 100, 999]:
    try:
        nova = locadora.renovarLocacao(codigo, hoje)
        print(f"{codigo}: renovada, devolução até
{nova.strftime('%d/%m/%Y')}")
    except ValueError as erro:
        print(f"{codigo}: {erro}")
```

```
Locações de Ana
Locação 100 | Câmera, Tripé | retirada 10/10/2026 | devolução até 15/10/2026
| atraso de 6 dias | multa R$ 24.00
Locação 102 | Microfone | retirada 19/10/2026 | devolução até 24/10/2026 | no
prazo | multa R$ 0.00
Total de multas: R$ 24.00
Locações de Bruno
Locação 101 | Projetor | retirada 18/10/2026 | devolução até 23/10/2026 | no
prazo | multa R$ 0.00
Total de multas: R$ 0.00
101: renovada, devolução até 28/10/2026
101: Limite de renovações atingido
100: Locação atrasada, não pode renovar
999: Locação inexistente
```

Dicas: um dicionário `{codigo: Locacao}` ajuda a agrupar as linhas do arquivo (`dict.get` devolve `None` quando a chave não existe). Procure cliente e equipamento pelo código em métodos próprios da `Locadora`. Use `timedelta(days=5)` para somar dias.

Para quem terminar cedo: grave as renovações de volta em arquivo, para que valham na próxima execução, e faça a `Locadora` receber a data atual como parâmetro opcional (`datetime.now()` por padrão).